Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách khoa
Khoa Khoa học và Kỹ thuật Máy tính



Học sâu và ứng dụng trong thị giác máy tính (CO5085)

Bài tập E1 – E3

Phân loại chữ số viết tay MNIST:
Softmax, MLP, CNN, Multi-Head Self-Attention, LSTM và GRU

Giảng viên: Thầy Lê Thành Sách
Thầy Nguyễn Quốc Minh

Thành viên nhóm: Đàm Quang Phục - 2570483
Hồ Bảo An - 2570164

Mã nguồn: `https://github.com/dqpinfo/Deep_Learning_Computer_Vision_CO5085`

Thành Phố Hồ Chí Minh, 11/2026

Mục lục

1 Quy định chung và dữ liệu **2**
1.1 Hình thức thực hiện . . . 2
1.2 Tập dữ liệu: MNIST . . . 2

2 E1 – Phân loại ảnh: Softmax, MLP, CNN **3**
2.1 Mục tiêu . . . 3
2.2 Kiến trúc . . . 3
2.3 Huấn luyện . . . 3
2.4 Kết quả . . . 3
2.5 Phân tích lỗi . . . 4
2.6 Nhận xét . . . 5

3 E2 – Transformer: MSA và tokenization **5**
3.1 Mục tiêu . . . 5
3.2 MSA tự viết . . . 5
3.3 Cách tạo token và bộ phân loại . . . 6
3.4 Kết quả . . . 6
3.5 Nhận xét . . . 6

4 E3 – LSTM / GRU trên chuỗi ảnh **7**
4.1 Mục tiêu . . . 7
4.2 Cách biến ảnh thành chuỗi . . . 7
4.3 Kiến trúc . . . 7
4.4 Huấn luyện . . . 8
4.5 Kết quả . . . 8
4.6 Phân tích lỗi . . . 8
4.7 Khả năng bắt phụ thuộc theo thời gian . . . 9
4.8 Nhận xét . . . 11

5 Tái lập kết quả **11**

6 Sử dụng công cụ AI **11**

Phiên bản W10 (hạn 09/11/2026): đủ E1, E2 và E3. Cả ba bài dùng chung tập **MNIST**. **E1** so sánh ba bộ phân loại Softmax, MLP và CNN: CNN đạt **98,91%** trên 10.000 ảnh test với số tham số ít hơn MLP (105.962 so với 109.386). **E2** tự cài đặt Multi-Head Self-Attention (MSA), đối chiếu với `nn.MultiheadAttention` và thử hai cách tạo token (theo patch và theo hàng). MSA tự viết cho kết quả **trùng khớp tuyệt đối** với bản PyTorch khi dùng chung trọng số; cách tạo token theo hàng tốt hơn theo patch khoảng 2 điểm phần trăm. **E3** đọc ảnh như một chuỗi 28 hàng bằng **LSTM** và **GRU**: GRU đạt **98,32%**, LSTM 98,20%, vượt MLP (96,97%) và gần CNN (98,91%) với ít tham số hơn cả hai. Mọi con số lấy từ `output/report_metrics.json` của từng bài.

1. Quy định chung và dữ liệu

1.1. Hình thức thực hiện

- Ba bài E1–E3 dùng **cùng một tập ảnh nhỏ** để các kết quả so sánh được với nhau.
- Vòng lặp huấn luyện được **tự viết bằng PyTorch** [6] trong `train.py` của mỗi bài: xoá gradient → forward → tính loss → `backward()` → `optimizer.step()`; không dùng API huấn luyện cấp cao.
- Mã nguồn tổ chức theo thư mục `Exercise_1/`, `Exercise_2/`, `Exercise_3/`, mỗi thư mục có `README.md` (môi trường, lệnh chạy), `dataset.py`, `models.py`, `train.py`, `evaluate.py`.

1.2. Tập dữ liệu: MNIST

Nhóm chọn **MNIST** [1]: ảnh chữ số viết tay 0–9, ảnh xám 28×28, 10 lớp, gồm 60.000 ảnh train và 10.000 ảnh test, gần cân bằng giữa các lớp. MNIST nằm trong danh sách tập được đề cho phép, nhỏ, tải nhanh qua `torchvision.datasets.MNIST` và huấn luyện được trên CPU. Nhờ vậy nhóm tập trung vào phần cài đặt mô hình (đặc biệt là MSA tự viết ở E2) thay vì chi phí tính toán. Tập được công bố bởi LeCun, Cortes và Burges, cho phép dùng tự do với yêu cầu trích dẫn.

Bảng 1: Cách dùng MNIST trong từng bài.

Bài	Tập train	Tập test	Tiền xử lý
E1	60.000 ảnh (toàn bộ)	10.000 ảnh (toàn bộ)	`ToTensor` + `Normalize(0.5, 0.5)` (pixel về $[-1, 1]$)
E2	5.000 ảnh đầu tiên	1.000 ảnh đầu tiên (protocol gốc) và 10.000 ảnh (đánh giá bổ sung)	`ToTensor` (pixel trong $[0, 1]$)
E3	60.000 ảnh (toàn bộ)	10.000 ảnh (toàn bộ)	Giống E1, để so sánh trực tiếp với MLP/CNN

E2 dùng tập con 5.000 ảnh train để huấn luyện nhanh nhiều cấu hình attention trên CPU. Thứ tự ảnh trong MNIST vốn đã ngẫu nhiên, nên lấy các ảnh đầu tiên vẫn cho tập con đủ 10 lớp. Vì 1.000 ảnh test khá ít (sai số chuẩn của accuracy khoảng ±1,1 điểm phần trăm), báo cáo đánh giá thêm trên toàn bộ 10.000 ảnh test để kết luận đáng tin hơn.

2. E1 – Phân loại ảnh: Softmax, MLP, CNN

2.1. Mục tiêu

Xây dựng và so sánh ba bộ phân loại trên MNIST: Softmax trên vector ảnh đã làm phẳng, MLP trên vector làm phẳng và CNN.

2.2. Kiến trúc

Bảng 2: Ba mô hình của E1 (`Exercise_1/models.py`). Mọi mô hình trả về logits 10 lớp; softmax nằm trong hàm loss.

Mô hình	Cấu trúc	Tham số
Softmax	Flatten $784 \to$ Linear$(784, 10)$	7.850
MLP	Flatten $784 \to$ Linear$(784, 128)$ + ReLU $\to$ Linear$(128, 64)$ + ReLU $\to$ Linear$(64, 10)$	109.386
CNN	[Conv 3×3, 16 kênh + BatchNorm + ReLU + MaxPool 2×2] $\to$ [Conv 3×3, 32 kênh + BatchNorm + ReLU + MaxPool 2×2] $\to$ Flatten $32 \times 7 \times 7 \to$ Linear$(1568, 64)$ + ReLU $\to$ Linear$(64, 10)$	105.962

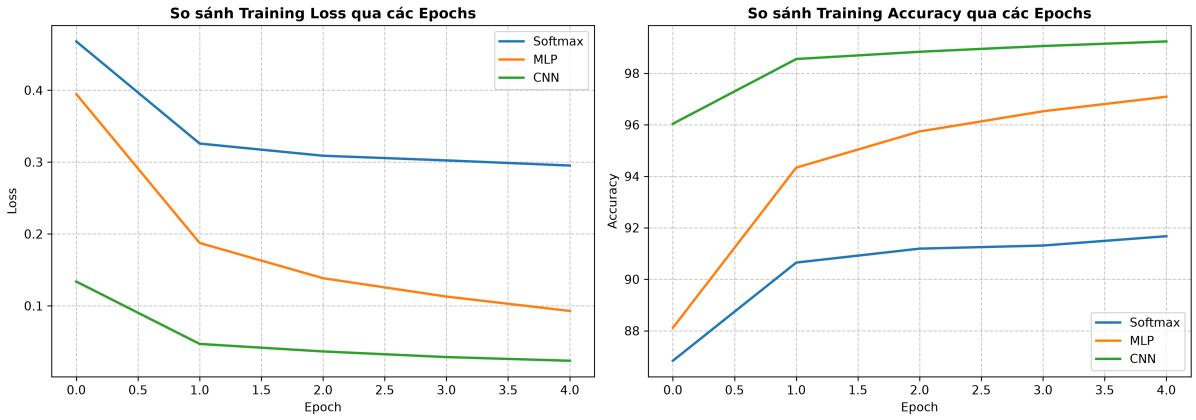
2.3. Huấn luyện

Cả ba mô hình dùng chung thiết lập: loss cross-entropy, optimizer Adam với learning rate 10^{-3}, batch 64, 5 epoch trên toàn bộ 60.000 ảnh train. Đánh giá trên 10.000 ảnh test bằng trọng số sau epoch cuối.

2.4. Kết quả

Bảng 3: Kết quả E1 trên 10.000 ảnh test MNIST. Thời gian đo trên CPU Apple M1.

Mô hình	Tham số	Train acc.	Test acc.	Macro-F1	Số ảnh sai	1 epoch train
Softmax	7.850	91,67%	91,08%	90,98%	892	2,5 giây
MLP	109.386	97,09%	96,97%	96,95%	303	2,7 giây
CNN	105.962	99,24%	**98,91%**	**98,90%**	**109**	12,7 giây

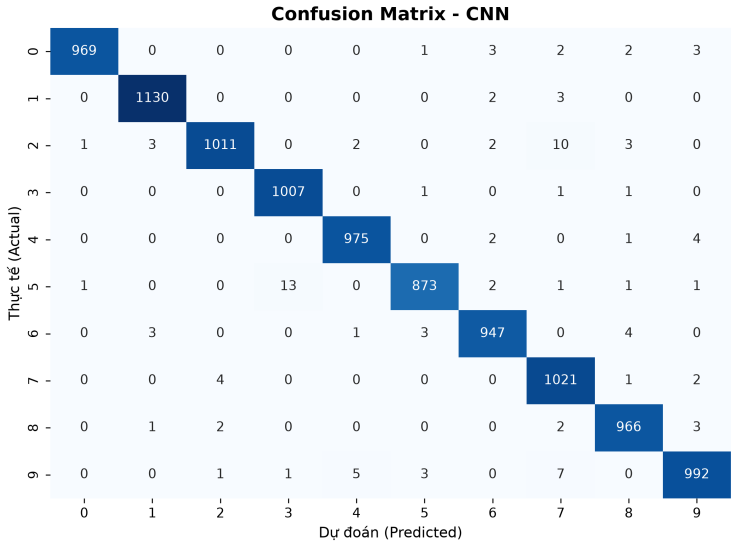


Hình 1: Loss và accuracy trên tập train qua 5 epoch (trục hoành đánh số epoch từ 0).

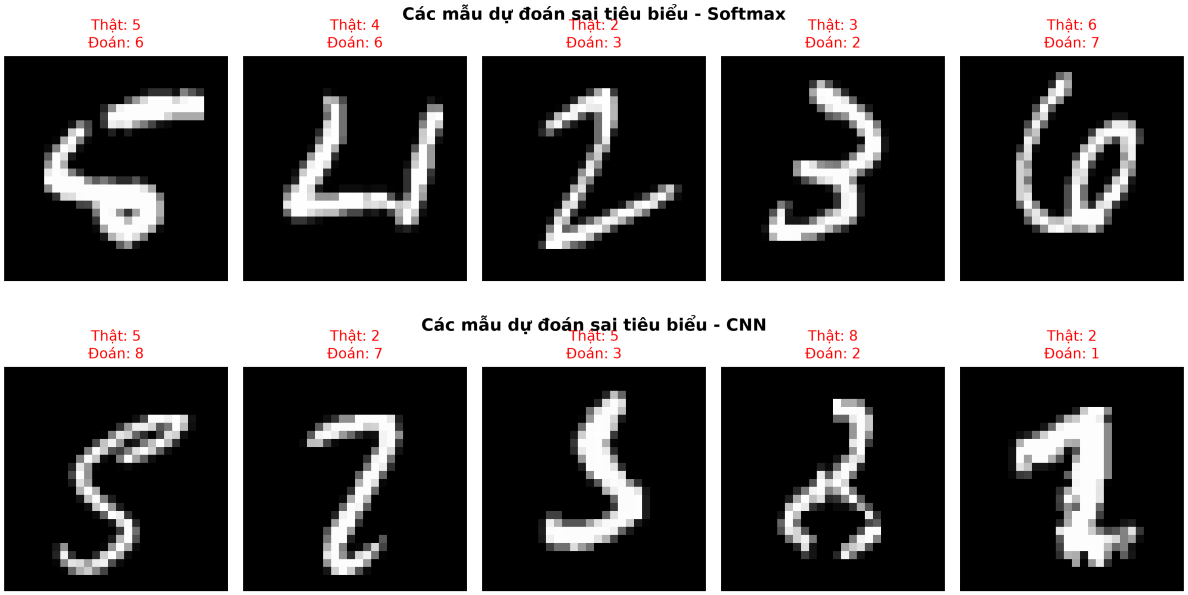
2.5. Phân tích lỗi

Bảng 4: Ba cặp nhầm lẫn thường gặp nhất của mỗi mô hình (nhãn thật → dự đoán, số ảnh).

Mô hình	Thứ 1	Thứ 2	Thứ 3
Softmax	2 → 8 (78)	3 → 5 (62)	4 → 9 (60)
MLP	7 → 9 (27)	9 → 4 (18)	8 → 9 (15)
CNN	5 → 3 (13)	2 → 7 (10)	9 → 7 (7)



Hình 2: Confusion matrix của CNN trên 10.000 ảnh test.



Hình 3: Một số ảnh bị đoán sai: Softmax (trên) và CNN (dưới).

2.6. Nhận xét

- **Thêm tầng ẩn phi tuyến giúp nhiều:** từ Softmax lên MLP, số ảnh sai giảm gần 3 lần (892 xuống 303). Softmax chỉ vẽ được ranh giới tuyến tính trên 784 pixel, nên nhầm giữa các chữ số có nét chồng lên nhau như 2/8, 3/5, 4/9.
- **CNN tốt nhất dù ít tham số hơn MLP:** 105.962 so với 109.386 tham số nhưng số ảnh sai chỉ còn khoảng một phần ba (109 so với 303). Tích chập dùng chung trọng số trên mọi vị trí và giữ cấu trúc không gian 2D, trong khi Softmax và MLP mất thông tin vị trí khi làm phẳng ảnh. Phần lớn tham số của CNN nằm ở lớp Linear$(1568, 64)$; hai lớp tích chập chỉ có khoảng 4.900 tham số.
- **Lỗi còn lại của CNN** chủ yếu là những ảnh mà người cũng dễ đọc nhầm: số 5 viết giống 3, số 2 có nét ngang ngắn giống 7 (Hình 3).
- **Không thấy overfitting rõ:** train accuracy và test accuracy chênh nhau dưới 0,6 điểm ở cả ba mô hình sau 5 epoch.
- **Chi phí:** một epoch của CNN chậm hơn khoảng 5 lần so với Softmax/MLP trên CPU, đổi lại sai số giảm mạnh.

Hạn chế. Bài chưa tách tập validation: mô hình được đánh giá trên test bằng trọng số sau epoch cuối, không dùng test để chọn siêu tham số. Đường cong học chỉ có số liệu trên tập train. `train.py` chưa cố định seed, nên chạy lại có thể lệch vài phần mười điểm phần trăm.

3. E2 – Transformer: MSA và tokenization

3.1. Mục tiêu

Hiện thực Multi-Head Self-Attention từ các phép toán cơ bản và bằng module của PyTorch, thử nhiều cách biến ảnh thành chuỗi token, rồi phân loại trên cùng tập MNIST của E1.

3.2. MSA tự viết

`MyMSA` (`Exercise_2/models.py`) cài đặt cơ chế attention của Transformer [2] và chỉ dùng `nn.Linear`, phép nhân ma trận và softmax. Với chuỗi token $X \in \mathbb{R}^{N \times C}$, $C = 64$ chiều và $h = 4$ head (mỗi head $d = C/h = 16$ chiều):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad \text{(tách thành } h \text{ head, mỗi head } d \text{ chiều)}$$

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d}}\right) V_i, \qquad \text{MSA}(X) = [\text{head}_1; \ldots; \text{head}_h]\, W_O$$

Bản đối chiếu `PyTorchMSA` gọi `nn.MultiheadAttention(64, 4, batch_first=True)`.

Kiểm tra tính đúng. Nhóm chép trọng số của `nn.MultiheadAttention` (`in_proj_weight` tách thành W_Q, W_K, W_V, cùng `out_proj`) sang `MyMSA` rồi so đầu ra trên một batch ngẫu nhiên kích thước $8 \times 17 \times 64$. Sai khác tuyệt đối lớn nhất là $\mathbf{0}$. Để chắc chắn phép kiểm tra có tác dụng, nhóm cộng thêm 10^{-3} vào một phần tử của W_K: sai khác tăng lên khoảng 10^{-4}. Vậy MSA tự viết tính **đúng cùng một hàm** với bản của PyTorch (`msa_equivalence_check` trong `report_metrics.py`).

3.3. Cách tạo token và bộ phân loại

Bảng 5: Hai cách biến ảnh 28×28 thành chuỗi token (embedding 64 chiều, có token CLS và positional embedding học được).

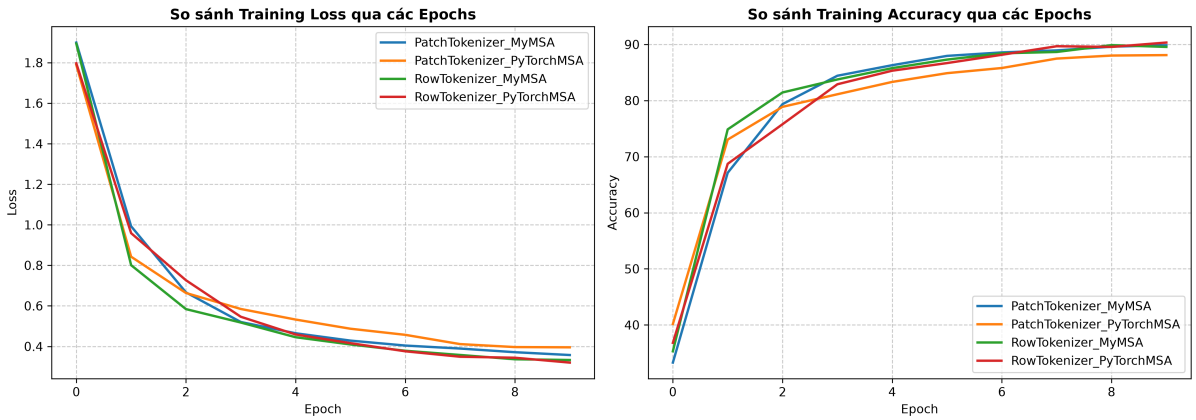
Tokenizer	Cách làm	Số token
Patch (kiểu ViT [3])	Chia ảnh thành lưới 4×4 ô, mỗi ô $7 \times 7 = 49$ pixel, chiếu tuyến tính lên 64 chiều	$16 + 1$ CLS
Row (hàng)	Mỗi hàng 28 pixel là một token, chiếu tuyến tính lên 64 chiều	$28 + 1$ CLS

Bộ phân loại `ImageTransformerClassifier` gồm: tokenizer → **một lớp MSA** → lấy embedding của token CLS → Linear$(64, 10)$. Mô hình cố ý giữ ở mức tối giản (không có LayerNorm, FFN hay kết nối tắt) để so sánh trực tiếp hai bản MSA và hai cách tạo token. Bốn cấu hình (2 tokenizer $\times$ 2 bản MSA) được huấn luyện giống nhau: Adam, learning rate 10^{-3}, batch 64, 10 epoch trên 5.000 ảnh train.

3.4. Kết quả

Bảng 6: Kết quả E2. Test 1k là protocol gốc của `evaluate.py`; test 10k là toàn bộ tập test MNIST. Cột cuối: thời gian một epoch huấn luyện (giây) trên CPU Apple M1.

Cấu hình	Tham số	Train acc.	Test 1k	Test 10k	F1 10k	Epoch (s)
Patch + MyMSA	21.642	89,86%	85,10%	86,67%	86,56%	0,41
Patch + PyTorch MSA	21.642	88,10%	85,70%	86,24%	86,05%	0,38
Row + MyMSA	21.066	89,56%	**88,90%**	**88,47%**	**88,18%**	0,41
Row + PyTorch MSA	21.066	90,34%	85,40%	88,15%	88,09%	0,42



Hình 4: Loss và accuracy trên tập train của 4 cấu hình qua 10 epoch.

3.5. Nhận xét

- **MSA tự viết và MSA của PyTorch tương đương:** cùng tokenizer, hai bản chênh nhau không quá 0,43 điểm trên 10.000 ảnh test. Khác biệt này đến từ khởi tạo ngẫu

nhiên và thứ tự batch, không phải từ cài đặt, vì phép kiểm tra ở mục 3.2 cho thấy hai bản tính cùng một hàm. Thời gian huấn luyện cũng gần như nhau.

- **Token theo hàng tốt hơn token theo patch** khoảng 1,5–2 điểm trên test 10k, với số tham số tương đương (khoảng 21 nghìn). Một lý do hợp lý: mỗi hàng giữ trọn một lát cắt ngang của nét chữ, và 28 token cho attention nhiều vị trí để kết hợp hơn 16 token. Patch 7×7 thì cắt nét chữ thành các mảnh nhỏ rời rạc.
- **Đánh giá trên 1.000 ảnh dễ gây hiểu nhầm:** trên test 1k, Row + PyTorch MSA (85,40%) có vẻ kém hẳn Row + MyMSA (88,90%), nhưng trên 10.000 ảnh hai cấu hình chỉ chênh 0,32 điểm. Vì vậy kết luận của nhóm dựa trên test 10k.
- **So với E1:** mô hình attention một lớp (khoảng 86–88%) kém xa CNN của E1 (98,91%). Nguyên nhân là mô hình chỉ có một lớp attention, không có FFN, chỉ học trên 5.000 ảnh, và attention không có sẵn "thiên kiến cục bộ" như tích chập nên cần nhiều dữ liệu hơn để học quan hệ giữa các vùng ảnh.

Hạn chế và việc tiếp theo. Đề khuyến nghị thử ba cách tạo token; nhóm hiện mới có hai (đạt mức tối thiểu). Cách thứ ba dự kiến là **CNN-stem** (dùng vài lớp tích chập rút feature map, mỗi vị trí là một token). Ngoài ra, nhóm có thể thêm LayerNorm, FFN và kết nối tắt để có một khối Transformer encoder đầy đủ, cố định seed, và tách tập validation.

4. E3 – LSTM / GRU trên chuỗi ảnh

4.1. Mục tiêu

Xem mỗi ảnh như một **chuỗi theo thời gian**, phân loại bằng mạng hồi quy LSTM [4] và GRU [5], rồi so sánh với hai baseline không hồi quy MLP và CNN của E1 trên cùng dữ liệu và cùng tiền xử lý.

4.2. Cách biến ảnh thành chuỗi

Nhóm dùng cách **mỗi hàng là một bước thời gian**: ảnh 28×28 trở thành chuỗi 28 bước, mỗi bước là vector 28 giá trị pixel của một hàng, đọc từ trên xuống dưới (`sequence_mode='row'` trong `Exercise_3/models.py`). Mạng phải "nhớ" các hàng đã đọc trong trạng thái ẩn để đến hàng cuối mới quyết định chữ số. Đề cho phép hai hướng: thử hai cách biểu diễn, hoặc so sánh LSTM với GRU trên cùng một cách biểu diễn. Nhóm chọn hướng thứ hai. Mã nguồn đã hỗ trợ sẵn cách đọc theo cột (`sequence_mode='col'`, hoán đổi hai chiều của ảnh) nhưng chưa được huấn luyện.

4.3. Kiến trúc

Bảng 7: Hai mô hình hồi quy của E3 (`ImageRNNClassifier` trong `Exercise_3/models.py`).

Mô hình	Cấu trúc	Tham số
LSTM	Chuỗi 28 bước × 28 chiều → `nn.LSTM`(28, 128, 1 lớp) → trạng thái ẩn ở bước cuối → Linear$(128, 10)$	82.186
GRU	Giống LSTM, thay bằng `nn.GRU`(28, 128, 1 lớp)	61.962

Một ô LSTM có 4 cổng (input, forget, output và ứng viên của ô nhớ) và giữ hai trạng thái: trạng thái ẩn h_t và ô nhớ c_t. GRU gộp lại còn 3 nhóm trọng số (cổng update, cổng reset, ứng viên) và chỉ có h_t. Vì vậy với cùng 128 chiều ẩn, GRU có khoảng 3/4 số tham số của

phần hồi quy: $3 \cdot 128 \cdot (28 + 128) + 768 = 60.672$ so với $4 \cdot 128 \cdot (28 + 128) + 1.024 = 80.896$; cộng lớp ra Linear$(128, 10)$ có 1.290 tham số.

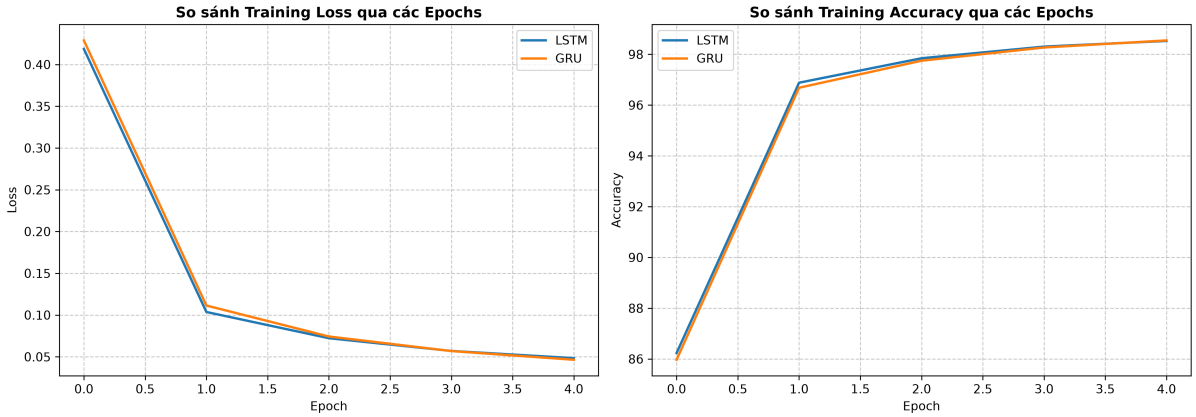
4.4. Huấn luyện

Giống hệt E1 để so sánh công bằng: cross-entropy, Adam với learning rate 10^{-3}, batch 64, 5 epoch trên 60.000 ảnh train, trạng thái ẩn khởi tạo bằng 0, đánh giá trên 10.000 ảnh test bằng trọng số sau epoch cuối (`Exercise_3/train.py`, vòng lặp tự viết).

4.5. Kết quả

Bảng 8: E3 so với baseline của E1 trên 10.000 ảnh test MNIST (cùng tiền xử lý, cùng thiết lập huấn luyện). Số liệu MLP/CNN lấy từ Bảng 3.

Mô hình	Tham số	Train acc.	Test acc.	Macro-F1	Số ảnh sai
MLP (E1)	109.386	97,09%	96,97%	96,95%	303
CNN (E1)	105.962	99,24%	**98,91%**	**98,90%**	**109**
LSTM	82.186	98,53%	98,20%	98,18%	180
GRU	**61.962**	98,55%	98,32%	98,30%	168



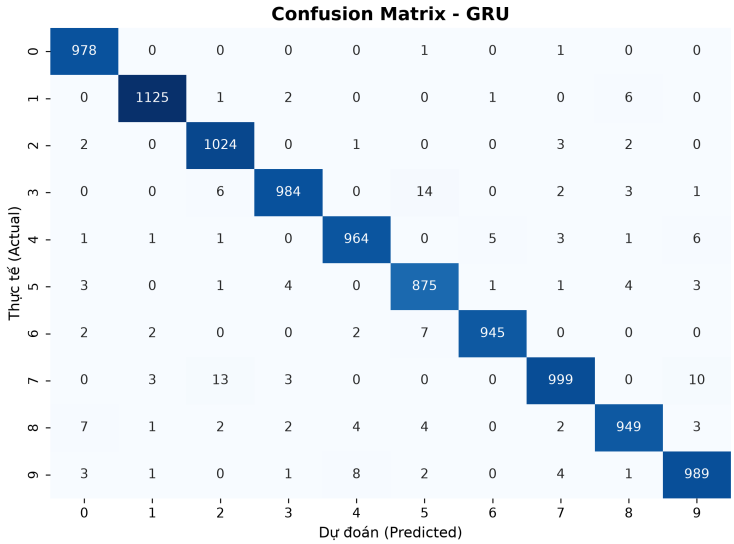
Hình 5: Loss và accuracy trên tập train của LSTM và GRU qua 5 epoch (trục hoành đánh số epoch từ 0).

Chi phí. Trên CPU Apple M1, một epoch huấn luyện mất 13,7 giây với LSTM và 11,2 giây với GRU; dự đoán toàn bộ 10.000 ảnh test mất 2,0 và 1,1 giây (MLP 0,8 giây, CNN 1,2 giây khi đo cùng lần). Thời gian phụ thuộc tải của máy lúc đo, chỉ nên đọc như bậc độ lớn. Mạng hồi quy phải xử lý 28 bước **tuần tự**, không song song theo chiều thời gian được, nên với số tham số ít hơn vẫn không nhanh hơn CNN.

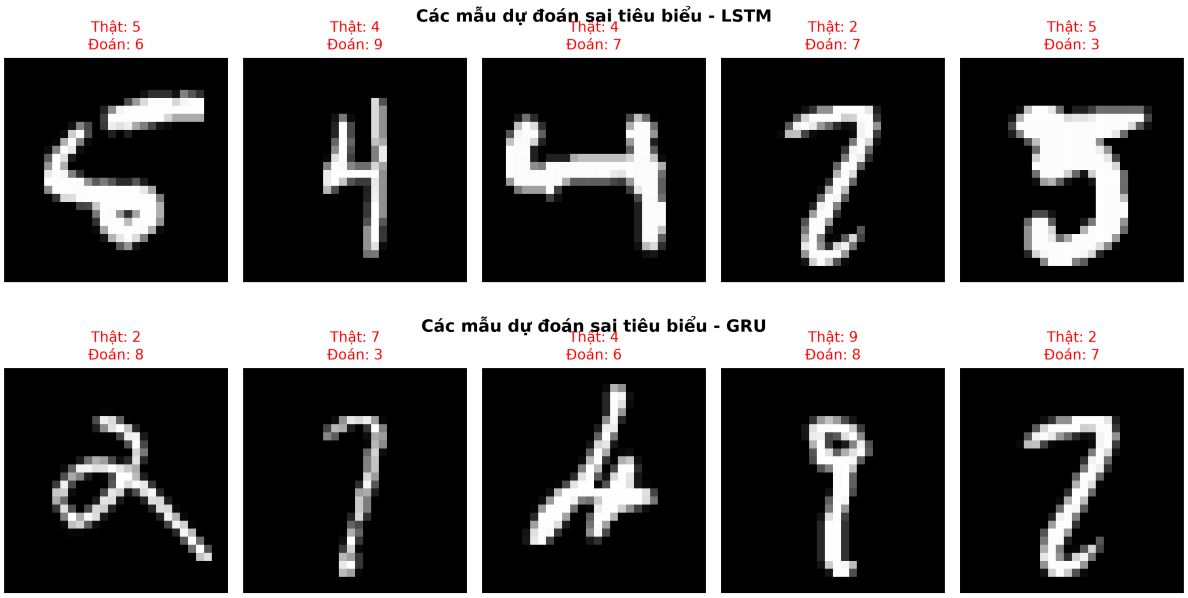
4.6. Phân tích lỗi

Bảng 9: Ba cặp nhầm lẫn thường gặp nhất (nhãn thật → dự đoán, số ảnh).

Mô hình	Thứ 1	Thứ 2	Thứ 3
LSTM	4 → 9 (14)	8 → 2 (11)	5 → 6 (10)
GRU	3 → 5 (14)	7 → 2 (13)	7 → 9 (10)



Hình 6: Confusion matrix của GRU trên 10.000 ảnh test.



Hình 7: Năm ảnh bị đoán sai đầu tiên trong tập test: LSTM (trên) và GRU (dưới).

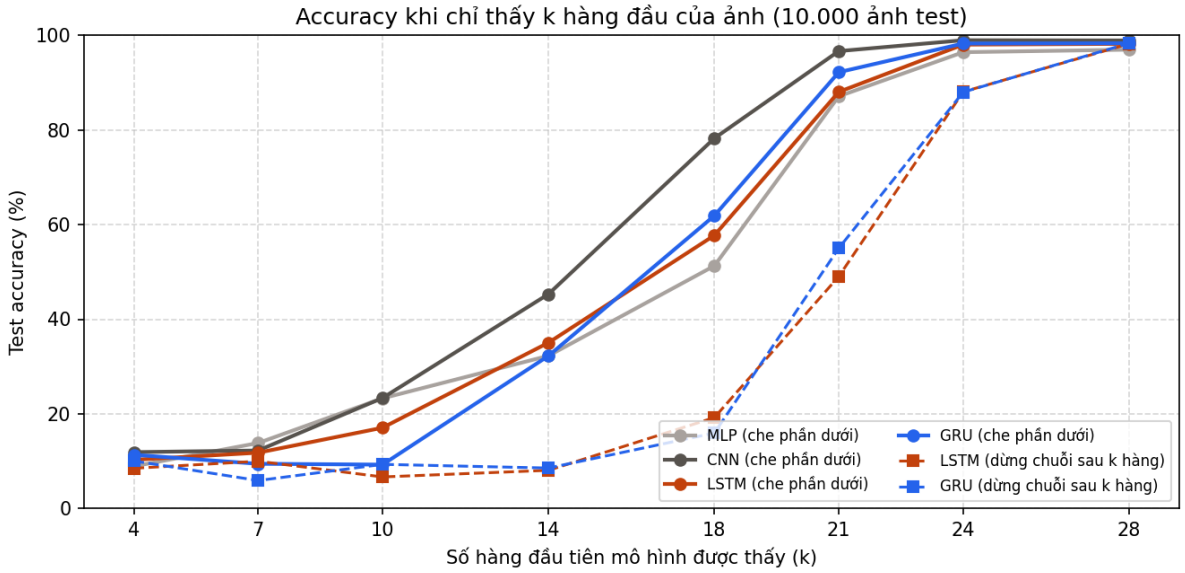
Các cặp nhầm của mạng hồi quy là những chữ số có phần trên hoặc phần dưới giống nhau khi đọc từng hàng: 4 và 9 cùng có phần trên khép kín hoặc gần khép, 7 và 2 cùng bắt đầu bằng một nét ngang. Lớp kém nhất là chữ số 5 với LSTM (96,6%) và chữ số 7 với GRU (97,2%).

4.7. Khả năng bắt phụ thuộc theo thời gian

Để thấy mỗi mô hình dựa vào phần nào của chuỗi, nhóm đo test accuracy khi mô hình chỉ được thấy k **hàng đầu** của ảnh, dùng trọng số đã huấn luyện, không học lại (`Exercise_3/report_metrics.py`). Có hai cách:

- **Che phần dưới:** giữ k hàng đầu, các hàng còn lại đặt về màu nền. Cả bốn mô hình nhận đủ ảnh 28×28; riêng LSTM/GRU vẫn đọc đủ 28 bước, trong đó $28 - k$ bước cuối là hàng trống.

- **Dừng chuỗi sau k hàng** (chỉ áp dụng cho LSTM/GRU): mạng chỉ đọc k bước rồi phân loại ngay.



Hình 8: Test accuracy theo số hàng đầu tiên mô hình được thấy (10.000 ảnh test).

Bảng 10: Số liệu của Hình 8 tại một số giá trị k (test accuracy, %).

Cách đo	$k = 10$	$k = 14$	$k = 18$	$k = 21$	$k = 24$	$k = 28$
MLP, che phần dưới	23,3	32,3	51,3	87,1	96,4	97,0
CNN, che phần dưới	23,4	45,3	78,2	96,7	99,0	98,9
LSTM, che phần dưới	17,1	35,0	57,7	88,0	98,0	98,2
GRU, che phần dưới	9,3	32,3	61,9	92,2	98,3	98,3
LSTM, dừng chuỗi	6,7	8,1	19,2	49,0	88,0	98,2
GRU, dừng chuỗi	9,3	8,6	16,0	55,0	88,0	98,3

- **Mạng hồi quy giữ được thông tin qua nhiều bước.** Khi che phần dưới, LSTM/GRU phải mang thông tin của k hàng đầu qua $28 - k$ bước trống mới tới bước cuối, nơi lớp phân loại đọc trạng thái ẩn. Với $k = 21$ (7 bước trống), GRU vẫn đúng 92,2% và LSTM 88,0%, tức trạng thái ẩn không "quên" phần đầu chữ số. Đây là phụ thuộc xa mà RNN thường gặp khó; cơ chế cổng của LSTM/GRU giải quyết được ở độ dài 28 bước này.
- **Nhưng mô hình chỉ "quyết định" ở cuối chuỗi.** Nếu dừng chuỗi sớm, accuracy giảm mạnh: tại $k = 21$ chỉ còn 49–55%, dù mạng đã thấy đúng các hàng như khi che phần dưới. Khi huấn luyện, mạng luôn đọc đủ 28 hàng và chữ số MNIST thường kết thúc bằng vài hàng trống, nên mạng học cách đưa ra quyết định sau khi đã đi qua đoạn trống ấy; trạng thái ẩn ở giữa chuỗi chưa phải là một dự đoán hoàn chỉnh.
- **So với baseline:** khi thấy một phần chữ số ($k = 18$), GRU (61,9%) và LSTM (57,7%) tốt hơn MLP (51,3%) nhưng kém CNN (78,2%). CNN nhận ra các nét cục bộ ở bất cứ đâu chúng xuất hiện, nên dùng phần nhìn thấy hiệu quả hơn. Với $k \leq 10$ mọi mô hình gần mức đoán ngẫu nhiên (10%) vì chữ số MNIST nằm ở vùng giữa ảnh, các hàng trên cùng gần như trống.

4.8. Nhận xét

- **Đọc ảnh theo hàng là cách biểu diễn hiệu quả:** cả LSTM và GRU vượt MLP (giảm từ 303 xuống 168–180 ảnh sai) với ít tham số hơn. MLP nhìn 784 pixel cùng lúc nhưng không biết pixel nào nằm cạnh pixel nào; RNN dùng chung một bộ trọng số cho mọi hàng và tích luỹ thông tin theo thứ tự trên xuống, gần với cách chia sẻ trọng số của CNN theo một chiều.
- **GRU nhỉnh hơn LSTM với ít tham số hơn 25%** (61.962 so với 82.186): 98,32% so với 98,20%, chênh 12 ảnh trên 10.000. Mức chênh này nhỏ, có thể đảo chiều khi chạy lại với seed khác; điều chắc chắn là với chuỗi ngắn 28 bước, ô nhớ riêng của LSTM không đem lại lợi thế. Hai đường cong học (Hình 5) gần như trùng nhau.
- **CNN vẫn tốt nhất** (109 ảnh sai), vì tích chập 2D khai thác cấu trúc không gian theo cả hai chiều, còn RNN theo hàng chỉ thấy quan hệ dọc thông qua trạng thái ẩn và quan hệ ngang trong một hàng qua một phép chiếu tuyến tính.
- **Không thấy overfitting:** train và test accuracy của LSTM/GRU chênh nhau khoảng 0,3 điểm sau 5 epoch.

Hạn chế. Nhóm mới thử một cách biểu diễn (theo hàng); cách theo cột đã có trong mã nhưng chưa huấn luyện. Như E1, bài chưa tách tập validation và `train.py` chưa cố định seed, nên mức chênh 0,12 điểm giữa GRU và LSTM chưa đủ để kết luận mô hình nào tốt hơn. Mỗi mô hình chỉ có một lớp hồi quy 128 chiều, không dùng LSTM hai chiều (bidirectional).

5. Tái lập kết quả

Trong mỗi thư mục bài tập (sau khi cài thư viện theo `README.md`):

- `python train.py`: huấn luyện và lưu trọng số, lịch sử train vào `output/`.
- `python evaluate.py`: đánh giá và vẽ biểu đồ, confusion matrix, ảnh lỗi.
- `python report_metrics.py`: tính các số liệu dùng trong báo cáo này từ trọng số đã lưu (không huấn luyện lại mô hình đã lưu; chỉ huấn luyện thử 1 epoch trên một mô hình mới để đo thời gian), kết quả ở `output/report_metrics.json`. Với E3, script này còn nạp trọng số MLP/CNN từ `Exercise_1/output/` để chạy thí nghiệm "chỉ thấy k hàng đầu" và vẽ `output/rows_seen.png`.

6. Sử dụng công cụ AI

Theo quy định liêm chính học thuật của môn học, nhóm ghi rõ: trợ lý AI (Claude) được dùng để hỗ trợ viết script thu thập số liệu, soạn bố cục báo cáo và rà soát lỗi. Nhóm đã đọc, chạy kiểm tra, chỉnh sửa và chịu trách nhiệm về toàn bộ nội dung; mọi mô hình, số liệu và nhận xét đều được nhóm hiểu và có thể giải thích.

Tài liệu

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 1998.

[2] A. Vaswani et al. Attention Is All You Need. *NeurIPS*, 2017.

[3] A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*, 2021.

[4] S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8), 1997.

[5] K. Cho et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP*, 2014.

[6] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 2019.